This version uses recursive division of the data rather than pre-calculating the division of the data into chunks, but it's a whole lot simpler than the previous version, and it's *still exception safe*. As before, you start by finding the length of the sequence ❶, and if it's smaller than the maximum chunk size, you resort to calling `std::accumulate` directly ❷. If there are more elements than your chunk size, you find the midpoint ❸ and then spawn an asynchronous task to handle that half ❹. The second half of the range is handled with a direct recursive call ❺, and then the results from the two chunks are added together ❻. The library ensures that the `std::async` calls make use of the hardware threads that are available without creating an overwhelming number of threads. Some of the "asynchronous" calls will actually be executed synchronously in the call to `get()` ❻.

The beauty of this is that not only can it take advantage of the hardware concurrency, but it's also trivially exception safe. If an exception is thrown by the recursive call ❺, the future created from the call to `std::async` ❹ will be destroyed as the exception propagates. This will in turn wait for the asynchronous task to finish, thus avoiding a dangling thread. On the other hand, if the asynchronous call throws, this is captured by the future, and the call to `get()` ❻ will rethrow the exception.

What other considerations do you need to take into account when designing concurrent code? Let's look at *scalability*. How much does the performance improve if you move your code to a system with more processors?

8.4.2. Scalability and Amdahl's law

Scalability is all about ensuring that your application can take advantage of additional processors in the system it's running on. At one extreme you have a single-threaded application that's completely unscalable; even if you add 100 processors to your system, the performance will remain unchanged. At the other extreme you have something like the SETI@Home[3] project, which is designed to take advantage of thousands of additional processors